

Booking Service Database Design

1. Overview

Purpose

The Booking Service is responsible for:

- Managing business availability
- Retrieving available booking slots
- Creating bookings
- Cancelling bookings
- Preventing double-bookings
- Publishing booking events

The Booking Service owns all scheduling and appointment data.

Responsibilities

Availability Management

Configure working hours
Store business schedules
Generate available slots
Retrieve availability

Booking Management

Create bookings
Cancel bookings
View bookings
Validate booking requests
Prevent double-bookings

Event Publishing

BOOKING_CREATED
BOOKING_CANCELLED

2. Database Technology

Property	Value
Database	PostgreSQL
Service	Booking Service
Tables	2
Aggregate Roots	Staff Availability, Booking

3. Why Two Tables?

KevaBook separates:

Availability

Defines when a business can accept appointments.

Example:

Emmanuel

Monday 09:00 - 17:00

Tuesday 09:00 - 17:00

Maxwell

Monday 10:00 - 18:00

Tuesday 10:00 - 18:00

Booking

Represents a confirmed appointment.

Example:

June 20

10:00 - 11:00

Staff

Maxwell

Booked by:

John Doe

Without this separation:

- Availability becomes difficult to manage.
- Slot generation becomes complicated.
- Booking validation becomes expensive.

This separation keeps:

- Availability management simple
- Slot generation efficient
- Booking validation fast
- Staff schedules independent

Aggregate 1: Availability

Table Name

staff_availability

Purpose

Stores recurring working schedules for individual staff members

Table Structure

Column	Type	Required	Description
availabilityId	Long	Yes	Unique primary key

staffId	Long	Yes	Staff member reference
dayOfWeek	Enum	Yes	Working days
startTime	Time	Yes	Opening time
endTime	Time	Yes	Closing time
isActive	Boolean	Yes	Availability status
createdAt	Date Time	Yes	Creation date timestamp
updatedAt	Date Time	Yes	Last date updated timestamp

Aggregate 2: Booking

Table Name

bookings

Purpose

Stores confirmed appointments.

Table Structure

Column	Type	Required	Description
bookingId	Long	Yes	Unique primary key
businessId	Long	Yes	Business reference
serviceId	Long	Yes	Service reference
clientName	String	Yes	Customer name
clientEmail	String	Yes	Customer email
clientPhoneNumber	String	Yes	Customer phone number

bookingDate	Date	Yes	Booking date
startTime	Time	Yes	Appointment start time
endTime	Time	Yes	Appointment end time
status	Enum	Yes	Booking status
createdAt	Date Time	Yes	Creation date timestamp
updatedAt	Date Time	Yes	Last update timestamp

Double-Booking Prevention

Business Rule

A staff member must never have two bookings occupying the same slot.

Example

Allowed

John

10:00 - 11:00 Haircut

Maxwell

10:00 - 11:00 Massage

Rejected

Emma

10:00 - 11:00 Haircut

John

10:00 - 11:00 Haircut

Availability Validation

Before creating a booking, the Booking Service validates:

1. Staff members exist.
2. A staff member is assigned to the requested service.

3. Staff members are available on the selected day.
4. Requested time falls within staff working hours.
5. No existing booking occupies the same slot for that staff member.

If the slot is already occupied, the system then returns a 409 conflict. An example of such response could be:

```
{  
  "code": "SLOT_NOT_AVAILABLE",  
  "message": "The selected time slot is already booked."  
}
```

Handling Concurrency Protection

Application-level validation alone is insufficient because multiple booking requests may arrive simultaneously.

For example:

Request A

John

10:00

Request B

John

10:00

Both requests may initially appear valid.

To prevent race conditions, the Booking Service uses a **database-level constraint** as the final protection layer.

Database Constraint

PostgreSQL enforces a unique booking slot for each business. This guarantees that only one booking can exist for a staff member at a specific date and start time.

Concurrent Booking Example:

Request A → Maxwell 10:00

Request B → John 10:00

Result:

Request A → Booking Created

Request B → Unique Constraint Violation

↓
409 Conflict

In such cases even if multiple requests arrive simultaneously, the database ensures that only one booking is successfully created.

Design Strategy

KevaBook uses a two-layer protection strategy to prevent double-booking:

Layer 1 – Application Validation

- Check slot availability.
- Provides immediate feedback to users.
- Reduces unnecessary database operations.

Layer 2 – Database Constraint

- Prevents race conditions.
- Guarantees data consistency.
- Acts as the final protection layer.

This approach ensures that booking availability remains accurate and that duplicate bookings cannot occur, even under concurrent user requests. KevaBook will use fixed booking slots generated from business availability and service duration. This allows the system to enforce uniqueness using businessId, booking date, start time. There is a room for more advanced overlap detection that can be introduced in future releases if arbitrary booking times are supported.

Data Ownership Rules

Booking Service Owns

Staff Availability

Booking

Booking Service References

Booking service references businessId and serviceId, staffId from Business Service and Service Catalog Service respectively. No foreign keys.

If you could add the db diagrams as you did for business services for the others. It would be great. The diagram should include the type of the column, nullability, and if it's PK.